7 String Handling (**Advanced**, grade level 4 – 5)

- This set of exercises includes more challenging program logic to further develop your programming skills.
- These exercises are *optional*, but if your aim is to specialize in software development, it is advisable to try to complete as many of these as possible.

Strings Strong Password

Write a program called **StringsStrongfPassword** that has a method called **checkStrength** that checks the strength of a password. The method should take a password as parameter and return ***true*** if the password is strong enough. Otherwise, it should return ***false***.

The main method first inputs a password from the user. Then it calls the method **checkStrength** with the inputted password. Finally, the main method prints "**OK**" if the password is strong enough. Otherwise it prints "**Not strong enough!**"

A password is considered to be strong enough if ***it has at least 8 characters*** and ***it includes characters from at least <u>three</u> different categories***. The four categories of characters are the following: uppercase letters, lowercase letters, digits, and other characters.

Example output:

```
Enter password: Carpe_diem
OK
```

Strings Decimal Part

Write a program called **StringsDecimalPart** that first inputs a decimal number from the user. Then the program determines how many digits there are after the decimal point in the decimal number. Finally, the program prints the result.

If the inputted value is not a "proper decimal number", the program should print "**Please enter a proper decimal number**".

Example output:

```
Enter a decimal number: 3,1415926535
10 decimal place(s)
```

Rules (in this exercise)

- A "proper decimal number" includes at least one digit in the integer part, a comma or dot as the decimal separator, and at least one digit in the fractional part.

Hints

- Most of the algorithm involves string handling. That is, you have to save the inputted string.
- The normal conversion from String to double can be used to validate the input. But, you need to do some extra checking to make sure that the input is a "proper decimal number".

- The **indexOf** method is very useful here.

Strings Space

Write a program called **StringsSpace** that has a method called **removeExtraSpace**. The method removes all extra whitespace from a string. The method should take a string as parameter, process the string content and return a new string.

The main method first inputs a string from the user and then calls the method **removeExtraSpace** with the inputted string. Finally, the main method prints the result enclosed in double quotes.

In this exercise, "extra whitespace" means leading/trailing whitespace and more than one whitespace character between two non-whitespace characters.

Example output:

```
Enter a string: 1      +       2    =       3
"1 + 2 = 3"
```

NOTE! Viope does not allow test cases with leading/trailing whitespace. Please make sure that your program works correctly with the following input:

```
Enter a string:     Hello!
"Hello!"
```

Strings Anagram or Not

Write a program called **StringsAnagramOrNot** that has a method called **isAnagram** that determines whether two texts are anagrams of each other. The method should take two strings as parameters and return **true** if the texts are anagrams of each other. Otherwise, the method should return **false**.

The main method first inputs two strings from the user and then calls the method **isAnagram** with the inputted strings. Finally, the main method prints either "**Anagram!**" or "**No anagram**".

Example output:

```
Enter first string: Astronomer
Enter second string: Moon starer!
Anagram!
```

Rules (in this exercise)

- The method **isAnagram** should allow only letters, whitespace, semicolons, colons, commas, dots, exclamation marks, question marks, hyphens, and quotes in a string. The string cannot contain an anagram if there are any other characters.
- The method should ignore the previously-mentioned punctuators when it determines if two texts are anagrams of each other.
- If there are no letters in the text, then the text cannot be an anagram.

Hints

- When some validation fails in a method, you can terminate the method execution immediately by executing a return statement.
- Some sorting might simplify your algorithm...

Strings Palindrome Sentence

Write a program called **StringsPalindromeSentence** that has a method called **isPalindrome** that determines whether a string contains a *palindrome*. The method should take a string as parameter and return ***true*** if the string contains a palindrome. Otherwise, the method should return ***false***.

The main method first inputs a string from the user and then calls the method **isPalindrome** with the inputted string. Finally, the main method prints either "**Palindrome!**" or "**No palindrome**".

Example output:

```
Enter a string: Was it a car or a cat I saw?
Palindrome!
```

Rules (in this exercise)

- The method **isPalindrome** should allow only letters, whitespace, commas, dots, semi-colons, question marks, exclamation marks, hyphens, and quotes in a palindrome. If there are any other characters, then the text cannot be a palindrome.
- If there are no letters in the text, the text cannot be a palindrome.